Today 7:25 AM

IoC concept

IoC (Inversion of Control) in Spring

1. What is IoC?

IoC (Inversion of Control) is a principle where **object creation, dependency management, and object lifecycle control are transferred from application code to a framework/container**.

Normally, your code controls object creation:

Java

```java
OrderService service = new OrderService();
PaymentService paymentService = new PaymentService();
```

With Spring IoC:

Java

```java
@Service
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

You don't create `PaymentService` manually. **Spring's IoC container creates it and injects it into** `OrderService` .

---

2. Why is it called "Inversion"?

Without IoC:

```
OrderService
```

```
      |
      | creates
      v
PaymentService
```

`OrderService` controls the creation of its dependency.

With IoC:

```
            Spring IoC Container
                    |
          creates and manages
                    |
        +-----------+-----------+
        |                       |
        v                       v
 OrderService              PaymentService
        |
        | receives dependency
        v
 PaymentService
```

The control is **inverted**:

```
Application Code
      ↓
creates/manages objects
```

becomes:

```
Spring Container
      ↓
creates/manages objects
      ↓
Application Code uses them
```

---

3. IoC vs Dependency Injection

These two concepts are related but **not exactly the same**.

IoC

IoC is the **broader principle**.

> Control of object creation and management is transferred from application code to a container/framework.

Dependency Injection

DI is the **common technique used to implement IoC**.

```java
@Service
public class OrderService {

    private final PaymentService paymentService;

    @Autowired
    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Here:

- **IoC** → Spring controls object creation/management.
- **DI** → Spring injects `PaymentService` into `OrderService`.

So:

```
IoC
 └── Dependency Injection
      ├── Constructor Injection
      ├── Setter Injection
      └── Field Injection
```

---

4. Spring IoC Container

Spring provides an **IoC container** that manages Spring beans.

The two important container interfaces are:

```
BeanFactory
    ↓
ApplicationContext
```

`ApplicationContext` is the container normally used in Spring Boot applications.

Example:

```java
ApplicationContext context =
```

```
        new AnnotationConfigApplicationContext(AppConfig.class);

OrderService service =
        context.getBean(OrderService.class);
```

The container is responsible for things such as:

- Bean creation
- Dependency injection
- Bean initialization
- Bean lifecycle
- Bean destruction
- Bean scope management
- Bean configuration
- Resolving dependencies
- Creating proxies when required

---

5. How IoC works internally

Consider:

Java

```java
@Service
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

and:

Java

```java
@Service
public class PaymentService {
}
```

During application startup, Spring roughly performs:

```
1. Component scanning
        ↓
2. Discover @Service classes
```

```
        ↓
3. Create BeanDefinitions
        ↓
4. Register BeanDefinitions
        ↓
5. Instantiate required beans
        ↓
6. Resolve dependencies
        ↓
7. Inject PaymentService into OrderService
        ↓
8. Initialize beans
        ↓
9. Store/manage beans in IoC container
```

Conceptually:

```
@Service
PaymentService
      ↓
BeanDefinition
      ↓
Spring Container
      ↓
PaymentService object
```

Then:

```
@Service
OrderService
      ↓
BeanDefinition
      ↓
Spring sees constructor dependency
      ↓
Find PaymentService bean
      ↓
Create OrderService(PaymentService)
```

---

6. IoC is more than dependency injection

A common interview mistake is saying:

> "IoC means dependency injection."

More accurately:

DI is one of the mechanisms through which Spring implements IoC.

Spring's IoC container also controls:

```
Object creation
      +
Dependency resolution
      +
Dependency injection
      +
Bean lifecycle
      +
Bean scope
      +
Bean destruction
      +
Proxy creation/interception
```

For example:

Java

```java
@Service
@Scope("prototype")
public class OrderService {
}
```

Spring can control the bean's scope.

Similarly:

Java

```java
@Transactional
public void processOrder() {
}
```

Spring may create/use a proxy around the bean to provide transaction behavior.

---

7. IoC Example Without Spring

Java

```java
public class OrderService {

    private PaymentService paymentService;

    public OrderService() {
        this.paymentService = new PaymentService();
```

```
    }
}
```

Problems:

- Tight coupling
- `OrderService` controls dependency creation
- Difficult to replace implementation
- Difficult to test
- Object lifecycle is manually managed

---

8. IoC Example With Spring

Java

```java
@Service
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Java

```java
@Service
public class PaymentService {
}
```

Now:

```
Spring
  |
  +-- creates PaymentService
  |
  +-- creates OrderService
  |
  +-- injects PaymentService
  |
  +-- manages both beans
```

`OrderService` no longer needs:

Java

```
new PaymentService();
```

This reduces coupling and improves testability.

---

9. IoC and loose coupling

IoC promotes **loose coupling**.

Prefer:

Java

```
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

rather than:

Java

```
public class OrderService {

    private final PaymentService paymentService =
            new PaymentService();
}
```

The first design allows Spring to provide different implementations.

For example:

Java

```
public interface PaymentService {
    void pay();
}
```

Java

```
@Service
```

```
public class CardPaymentService implements PaymentService {
    public void pay() {
    }
}
```

Java

```
@Service
public class UpiPaymentService implements PaymentService {
    public void pay() {
    }
}
```

Spring can resolve the appropriate implementation using mechanisms such as:

```
@Qualifier
@Primary
Collection injection
```

---

10. IoC in Spring Boot

In Spring Boot, you normally don't explicitly create the container.

Java

```
@SpringBootApplication
public class Application {

    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}
```

`SpringApplication.run()` bootstraps the Spring application context.

Conceptually:

```
SpringApplication.run()
        ↓
Create ApplicationContext
        ↓
Load configuration
        ↓
Component scanning
        ↓
Register BeanDefinitions
        ↓
```

```
Create beans
        ↓
Resolve dependencies
        ↓
Initialize beans
        ↓
Application ready
```

---

11. Important interview distinction

IoC

Principle

The control of object creation and management is inverted from application code to the framework/container.

IoC Container

Implementation mechanism in Spring

The Spring container manages beans and their dependencies/lifecycle.

Dependency Injection

Technique

Dependencies are supplied to an object by the container instead of the object creating them itself.

Spring

```
IoC Principle
      ↓
Spring IoC Container
      ↓
Bean Management
      ↓
Dependency Injection
      ↓
Constructor / Setter / Field Injection
```

What you should know at 7–8 years experience

For interviews, focus on these IoC areas:

1. **IoC vs DI**
2. **BeanFactory vs ApplicationContext**
3. **How Spring IoC container works**
4. **BeanDefinition**
5. **Bean registration**
6. **Bean creation and instantiation**
7. **Dependency resolution**
8. **Constructor injection**
9. **Bean lifecycle**
10. **Singleton/prototype scopes**
11. **Circular dependencies**
12. **Lazy vs eager initialization**
13. **BeanPostProcessor**
14. **FactoryBean / Factory methods**
15. **Proxy creation and how IoC interacts with AOP**

The key statement to remember for an interview is:

> **IoC is a design principle where control of object creation, dependency management, and lifecycle is transferred from application code to the Spring IoC container. Dependency Injection is the primary mechanism Spring uses to implement IoC.**

Dependency Injection (DI)

1. What is Dependency Injection?

Dependency Injection (DI) is a design technique where an object's dependencies are **provided to it from outside**, instead of the object creating those dependencies itself.

Without DI

Java

```java
public class OrderService {

    private PaymentService paymentService;

    public OrderService() {
        this.paymentService = new PaymentService();
    }
}
```

`OrderService` creates its own dependency.

```
OrderService
     |
     | new
     ↓
PaymentService
```

With DI

Java

```java
@Service
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Now `PaymentService` is supplied from outside.

```
Spring Container
       |
       | creates
       ↓
PaymentService
       |
       | injects
       ↓
```

```
OrderService
```

---

2. What is a Dependency?

A **dependency** is an object that another object requires to perform its work.

Example:

```java
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Here:

```
OrderService → depends on → PaymentService
```

Therefore, `PaymentService` is a dependency of `OrderService`.

---

3. Why do we need DI?

Without DI:

```java
public class OrderService {

    private PaymentService paymentService =
            new PaymentService();
}
```

Problems:

- Tight coupling
- Difficult to replace implementation
- Difficult unit testing
- Object creation logic is mixed with business logic
- Difficult to manage object lifecycle

- Difficult to configure complex dependency graphs

With DI:

```java
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

`OrderService` only says:

"I need a `PaymentService`."

It doesn't care how it is created.

---

4. DI in Spring

Spring implements DI through its **IoC Container**.

Example:

```java
@Service
public class PaymentService {
}
```

```java
@Service
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

During startup, Spring:

```
1. Finds PaymentService
        ↓
2. Creates PaymentService bean
        ↓
3. Finds OrderService
        ↓
4. Inspects its constructor
        ↓
5. Resolves PaymentService
        ↓
6. Creates OrderService
        ↓
7. Passes PaymentService to constructor
```

Conceptually:

```java
PaymentService paymentService =
        new PaymentService();

OrderService orderService =
        new OrderService(paymentService);
```

The important difference is that **Spring controls this process**.

---

5. Types of Dependency Injection in Spring

There are three commonly discussed forms:

```
Dependency Injection
│
├── Constructor Injection     ← Preferred
├── Setter Injection
└── Field Injection           ← Generally discouraged
```

5.1 Constructor Injection

```java
@Service
public class OrderService {

    private final PaymentService paymentService;
```

```
    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Spring provides the dependency through the constructor.

Advantages

- Dependency is mandatory and explicit
- Supports immutable fields
- Easy unit testing
- Object cannot normally exist without required dependencies
- Helps identify excessive dependencies
- Recommended approach in modern Spring

With a single constructor, `@Autowired` is not required:

Java

```java
public OrderService(PaymentService paymentService) {
    this.paymentService = paymentService;
}
```

---

6. Setter Injection

Java

```java
@Service
public class OrderService {

    private PaymentService paymentService;

    @Autowired
    public void setPaymentService(
            PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

The dependency is supplied through a setter.

Useful when the dependency is **optional or can be changed after construction**, although constructor injection is generally preferred for required dependencies.

---

7. Field Injection

```java
@Service
public class OrderService {

    @Autowired
    private PaymentService paymentService;
}
```

Spring injects directly into the field.

Although this works, it is generally discouraged for application code because:

- Dependencies aren't explicit in the constructor
- Fields cannot easily be `final`
- Unit testing is less straightforward
- Objects can be created in an incompletely initialized state
- Dependency requirements are hidden

Prefer:

```java
public OrderService(PaymentService paymentService) {
    this.paymentService = paymentService;
}
```

---

8. DI with Interfaces

DI becomes particularly useful when programming against abstractions.

```java
public interface PaymentService {
    void pay();
}
```

Implementations:

```java
@Service
public class CardPaymentService
```

```
        implements PaymentService {

    @Override
    public void pay() {
    }
}
```

Java

```
@Service
public class UpiPaymentService
        implements PaymentService {

    @Override
    public void pay() {
    }
}
```

Consumer:

Java

```
@Service
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Now there are multiple candidates.

Spring needs to determine which implementation to inject.

Common mechanisms:

```
@Primary
@Qualifier
Collection injection
```

For example:

Java

```
@Service
@Primary
public class CardPaymentService
        implements PaymentService {
```

```
}
```

Or:

```java
public OrderService(
        @Qualifier("upiPaymentService")
        PaymentService paymentService) {
    this.paymentService = paymentService;
}
```

---

9. DI and Loose Coupling

Consider:

```java
public class OrderService {

    private final CardPaymentService paymentService;

    public OrderService() {
        this.paymentService =
                new CardPaymentService();
    }
}
```

This is tightly coupled to:

```
CardPaymentService
```

With DI:

```java
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Now:

```
OrderService
     |
     ↓
PaymentService interface
     ↑
     |
+----+----------------+
|                     |
CardPaymentService  UpiPaymentService
```

The implementation can be changed without modifying `OrderService` .

---

10. DI and Unit Testing

One of the biggest practical benefits of DI is testability.

Production:

Java

```java
PaymentService paymentService =
        new RealPaymentService();

OrderService orderService =
        new OrderService(paymentService);
```

Test:

Java

```java
PaymentService paymentService =
        mock(PaymentService.class);

OrderService orderService =
        new OrderService(paymentService);
```

The class doesn't need to know whether it received a real implementation or a mock.

---

11. DI vs IoC

These are related but should not be treated as synonyms.

IoC

A **principle**:

> Control of object creation and management is transferred from application code to the framework/container.

DI

A **technique**:

> Dependencies are supplied to an object instead of the object creating them itself.

Relationship:

```
IoC
 ↓
Spring IoC Container
 ↓
Dependency Injection
 ↓
Constructor / Setter / Field Injection
```

---

12. DI vs Service Locator

This is an important architectural distinction.

Dependency Injection

The dependency is explicitly supplied:

```java
public OrderService(PaymentService paymentService) {
    this.paymentService = paymentService;
}
```

Service Locator

The class asks the container for the dependency:

```java
PaymentService paymentService =
        context.getBean(PaymentService.class);
```

DI is generally preferred because dependencies are explicit and the class doesn't depend directly on the container.

---

13. DI lifecycle perspective

For a typical singleton bean:

```
Application startup
       ↓
BeanDefinition created
       ↓
BeanDefinition registered
       ↓
Bean instantiated
       ↓
Dependencies resolved
       ↓
Dependencies injected
       ↓
BeanPostProcessor processing
       ↓
Initialization
       ↓
Bean ready
```

For constructor injection specifically, dependency resolution happens **as part of constructing the object**:

```
Need OrderService
       ↓
Resolve PaymentService
       ↓
Create/get PaymentService
       ↓
Call OrderService(PaymentService)
       ↓
OrderService instance created
```

So there isn't an already-created `OrderService` object waiting for constructor injection. **The constructor dependency is supplied while the `OrderService` object is being constructed.**

---

14. Key points for a 7–8 year developer

You should be able to explain:

1. **What DI is**
2. **Why DI is needed**
3. **IoC vs DI**

4. **Constructor vs Setter vs Field injection**
5. **Why constructor injection is preferred**
6. **How Spring resolves dependencies**
7. **Multiple candidate resolution**
   - `@Primary`
   - `@Qualifier`
   - Collection injection
8. **Dependency resolution with interfaces**
9. **DI and loose coupling**
10. **DI and unit testing**
11. **Circular dependencies**
12. **DI with lazy/prototype beans**
13. **How constructor injection participates in bean creation**
14. **Difference between DI and Service Locator**

Interview definition

> **Dependency Injection is a technique where an object's dependencies are provided by an external component, typically the Spring IoC container, rather than being created by the object itself. In Spring, DI is primarily performed through constructor injection, setter injection, or field injection, with constructor injection being the recommended approach for required dependencies.**

Benefits of IoC

IoC (Inversion of Control) provides several important benefits by transferring object creation, dependency management, and lifecycle management from application code to the Spring container.

1. Loose Coupling

Without IoC:

Java

```java
public class OrderService {
```

```
    private PaymentService paymentService =
            new PaymentService();
}
```

`OrderService` directly controls the creation of `PaymentService`.

With IoC:

Java

```java
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Spring provides the dependency.

```
OrderService
     |
     ↓
PaymentService
```

The consumer doesn't control how the dependency is created.

Benefit: Components become less tightly coupled.

---

2. Dependency Management

Without IoC, developers manually create dependency graphs:

```
OrderService
   ↓
PaymentService
   ↓
PaymentGateway
   ↓
HttpClient
```

With Spring:

```
            Spring Container
                   |
        manages dependency graph
```

```
                    |
      +-------------+-------------+
      ↓             ↓             ↓
OrderService   PaymentService   PaymentGateway
```

Spring resolves and injects dependencies automatically.

Benefit: Complex dependency management becomes centralized.

---

3. Better Testability

IoC makes dependencies easy to replace during testing.

Java

```java
PaymentService mockPaymentService =
        mock(PaymentService.class);

OrderService service =
        new OrderService(mockPaymentService);
```

You can replace real implementations with mocks, stubs, or test implementations.

Benefit: Easier and cleaner unit testing.

---

4. Easier Implementation Replacement

Suppose:

Java

```java
public interface PaymentService {
    void pay();
}
```

Implementations:

```
PaymentService
    ├── CardPaymentService
    ├── UpiPaymentService
    └── WalletPaymentService
```

The consuming class depends on the abstraction:

Java

```java
public OrderService(PaymentService paymentService) {
```

```
    this.paymentService = paymentService;
}
```

Spring can select the implementation using:

- `@Primary`
- `@Qualifier`
- Configuration
- Profiles

Benefit: Implementations can be changed with minimal changes to business code.

---

5. Centralized Object Creation

Without IoC:

Java

```java
new OrderService();
new PaymentService();
new NotificationService();
new UserRepository();
```

Object creation can be scattered throughout the application.

With IoC:

```
            Spring Container
                   |
       Object creation/management
                   |
   +-------+-------+-------+-------+
   ↓       ↓       ↓       ↓
Order   Payment  Notify   User
```

Benefit: Object creation is centralized and consistently managed.

---

6. Lifecycle Management

Spring controls bean lifecycle:

```
Instantiation
      ↓
Dependency Injection
      ↓
```

```
Initialization
      ↓
BeanPostProcessor
      ↓
Bean Ready
      ↓
Application Shutdown
      ↓
Destruction
```

You don't need to manually manage the lifecycle of every Spring bean.

Benefit: Consistent lifecycle handling and cleanup.

---

7. Scope Management

Spring can control how long and how many instances of a bean exist.

Common scopes:

```
singleton
prototype
request
session
application
websocket
```

For example:

Java

```java
@Scope("prototype")
@Service
public class OrderContext {
}
```

Benefit: Object lifetime and instance creation policies can be managed declaratively.

---

8. Promotes Single Responsibility

A business class can focus on business logic instead of:

```
Creating dependencies
Managing dependencies
Managing object lifecycle
Managing configuration
```

For example:

```java
@Service
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }

    public void placeOrder() {
        paymentService.pay();
    }
}
```

`OrderService` focuses on order processing.

Benefit: Cleaner separation of responsibilities.

---

9. Supports Open/Closed Principle

IoC combined with interfaces and DI makes it easier to extend behavior without modifying existing consumers.

```
PaymentService
      ↑
      |
+-----+----------------+
|                      |
CardPaymentService   UpiPaymentService
```

You can introduce another implementation:

```
WalletPaymentService
```

without changing the consumer's dependency type.

Benefit: Easier extension and reduced modification of existing code.

---

10. Centralized Configuration

Spring can control which implementation should be injected using:

Java

```java
@Primary
```

Java

```java
@Qualifier
```

or configuration:

Java

```java
@Bean
public PaymentService paymentService() {
    return new CardPaymentService();
}
```

This separates configuration decisions from business logic.

IoC Benefits — Interview Summary

The most important benefits to remember:

Benefit	What IoC provides
Loose coupling	Removes direct dependency creation
Dependency management	Container resolves dependency graph
Testability	Easy replacement with mocks/stubs
Flexibility	Implementations can be changed easily
Lifecycle management	Container manages bean lifecycle
Scope management	Controls bean instance/lifetime
Separation of concerns	Business code doesn't manage infrastructure
Centralized configuration	Object creation/configuration is centralized
Extensibility	Works well with abstractions and DI

Strong interview answer

The primary benefit of IoC is that it separates object creation and dependency management from business logic. The Spring container manages bean creation, dependency injection, lifecycle, scope, and configuration. This results in loosely coupled, testable, maintainable, and flexible applications.

- IoC container startup

IoC Container Startup

In Spring Boot, **IoC container startup** is the process through which Spring creates and prepares the `ApplicationContext`, discovers bean definitions, creates beans, resolves dependencies, initializes them, and finally makes the application ready.

The key entry point is:

Java

```java
@SpringBootApplication
public class Application {

    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}
```

---

1. High-Level Startup Flow

For a typical Spring Boot application:

```
SpringApplication.run()
        ↓
Create / prepare ApplicationContext
        ↓
Load Environment & Configuration
        ↓
Process @SpringBootApplication
        ↓
Component Scanning
        ↓
Create BeanDefinitions
```

```
        ↓
Register BeanDefinitions
        ↓
Run BeanFactoryPostProcessors
        ↓
Instantiate non-lazy singleton beans
        ↓
Resolve dependencies
        ↓
Dependency Injection
        ↓
BeanPostProcessors
        ↓
Bean Initialization
        ↓
ApplicationContext refreshed
        ↓
ApplicationReadyEvent
        ↓
Application Ready
```

---

2. `SpringApplication.run()`

Everything starts here:

‹/› Java

```java
SpringApplication.run(Application.class, args);
```

Spring Boot uses `SpringApplication` to bootstrap the application.

Conceptually:

```
main()
  ↓
SpringApplication.run()
  ↓
Spring Boot startup
  ↓
ApplicationContext
```

Spring determines things such as:

- Which type of `ApplicationContext` to create
- Application configuration
- Environment and properties
- Auto-configuration

- Component scanning
- Bean registration

---

3. ApplicationContext Creation

Spring creates an appropriate `ApplicationContext`.

For a typical Spring Boot web application, this will be a web-aware application context.

Conceptually:

```
SpringApplication
       ↓
ApplicationContext
       ↓
BeanFactory
       ↓
BeanDefinitionRegistry
```

The `ApplicationContext` is the central container through which Spring manages application beans.

---

4. Environment and Configuration

Spring prepares the application's environment.

It loads configuration from sources such as:

```
application.properties
application.yml
Environment variables
System properties
Command-line arguments
Profiles
```

For example:

‹/› YAML

```yaml
server:
  port: 8080

spring:
  profiles:
```

```
    active: prod
```

These values become available through Spring's `Environment`.

---

5. Processing `@SpringBootApplication`

A typical application has:

```java
@SpringBootApplication
public class Application {
}
```

`@SpringBootApplication` is effectively a combination of:

```java
@Configuration
@EnableAutoConfiguration
@ComponentScan
```

Therefore, it tells Spring:

```
@Configuration
       ↓
This class can provide configuration

@ComponentScan
       ↓
Find application components

@EnableAutoConfiguration
       ↓
Apply appropriate Spring Boot auto-configurations
```

---

6. Component Scanning

Spring scans configured packages for components such as:

```java
@Component
@Service
@Repository
```

```
@Controller
@RestController
@Configuration
```

Example:

Java

```
@Service
public class OrderService {
}
```

Spring discovers `OrderService`.

But at this point, it is important to understand:

> **Discovering the class is not the same as creating the object.**

Spring first creates metadata describing the bean.

---

7. BeanDefinition Creation

For discovered components, Spring creates a **BeanDefinition**.

For:

Java

```
@Service
public class OrderService {
}
```

Spring creates metadata conceptually similar to:

```
BeanDefinition
-------------------------
Bean class: OrderService
Scope: singleton
Lazy: false
Autowire information: ...
Init method: ...
Destroy method: ...
```

This metadata tells Spring **how the bean should be managed**.

---

8. BeanDefinition Registration

The BeanDefinition is registered with the container.

Conceptually:

```
OrderService.class
      ↓
BeanDefinition
      ↓
BeanDefinitionRegistry
      ↓
ApplicationContext
```

The container now knows:

> "There is a bean called `orderService`, and this is how it should be created."

The same happens for other beans.

---

9. Configuration Classes and `@Bean`

Spring also processes configuration classes.

Example:

Java

```java
@Configuration
public class AppConfig {

    @Bean
    public PaymentService paymentService() {
        return new PaymentService();
    }
}
```

The `@Bean` method contributes a BeanDefinition to the container.

So beans can come from multiple sources:

```
@Component / @Service / @Repository
              ↓
        Component Scan

@Bean
              ↓
```

```
     Configuration Processing

Auto-configuration
              ↓
     Spring Boot Configuration
```

All ultimately result in bean definitions managed by the container.

---

10. BeanFactoryPostProcessor Phase

After BeanDefinitions have been registered, Spring can modify/process the **bean metadata** before actual bean instantiation.

This is the role of:

Java

```
BeanFactoryPostProcessor
```

Important distinction:

```
BeanFactoryPostProcessor
        ↓
Works primarily on BeanDefinitions / metadata
```

Whereas:

```
BeanPostProcessor
        ↓
Works on bean instances
```

This distinction is frequently asked in interviews.

---

11. Singleton Bean Creation

After the container has been prepared, Spring creates the required **non-lazy singleton beans** during startup.

For example:

Java

```
@Service
public class PaymentService {
```

Spring creates the instance.

Conceptually:

```
BeanDefinition
      ↓
Instantiation
      ↓
PaymentService object
```

---

12. Dependency Resolution

Suppose:

```java
@Service
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Spring sees:

```
OrderService
      |
      | requires
      ↓
PaymentService
```

The container resolves the dependency.

If `PaymentService` isn't created yet, Spring creates/resolves it first.

Conceptually:

```
Create OrderService
       ↓
Need PaymentService
       ↓
Resolve PaymentService
```

```
        ↓
Create/get PaymentService
        ↓
Inject into OrderService constructor
        ↓
OrderService created
```

---

13. Dependency Injection

For constructor injection:

Java

```java
public OrderService(PaymentService paymentService) {
    this.paymentService = paymentService;
}
```

The dependency is supplied **during object construction**.

Conceptually:

Java

```java
PaymentService paymentService =
        getBean(PaymentService.class);

OrderService orderService =
        new OrderService(paymentService);
```

This is conceptual behavior, not the exact internal implementation.

---

14. BeanPostProcessor

Once the bean instance exists, Spring applies `BeanPostProcessor`s.

Conceptually:

```
Bean instance
     ↓
BeanPostProcessor
     ↓
Initialization
     ↓
BeanPostProcessor
     ↓
Possibly proxy
```

```
     ↓
Final bean
```

Examples of Spring functionality that can involve post-processing include:

- `@Autowired`
- `@Value`
- AOP
- `@Transactional`
- `@Async`

For example, a transactional bean may ultimately be exposed through a proxy.

---

15. Bean Initialization

Spring then performs bean initialization.

Common lifecycle mechanisms include:

‹/› Java

```java
@PostConstruct
```

‹/› Java

```java
InitializingBean
```

‹/› Java

```java
@Bean(initMethod = "init")
```

Example:

‹/› Java

```java
@PostConstruct
public void init() {
    // initialization logic
}
```

Conceptually:

```
Instantiation
     ↓
```

```
Dependency Injection
      ↓
@PostConstruct
      ↓
Initialization callbacks
      ↓
Bean ready
```

---

16. `ApplicationContext` Refresh

The **refresh phase** is a major part of Spring container startup.

`ApplicationContext.refresh()` coordinates much of the container initialization process.

At a high level:

```
prepare context
      ↓
load/register bean definitions
      ↓
prepare BeanFactory
      ↓
register BeanPostProcessors
      ↓
initialize infrastructure
      ↓
create non-lazy singleton beans
      ↓
finish refresh
```

The exact internal sequence is more detailed, but this is the important conceptual flow for interviews.

---

17. Application Events

Spring publishes lifecycle events during startup.

Examples include:

```
ApplicationStartingEvent
ApplicationEnvironmentPreparedEvent
ApplicationContextInitializedEvent
ApplicationPreparedEvent
ContextRefreshedEvent
ApplicationStartedEvent
```

The most important distinction:

`ApplicationStartedEvent`

The application context has started, but startup runners may still need to execute.

`ApplicationReadyEvent`

The application is considered ready to serve requests.

You can listen for it:

Java

```java
@Component
public class StartupListener {

    @EventListener
    public void onReady(ApplicationReadyEvent event) {
        System.out.println("Application is ready");
    }
}
```

---

18. What happens with `@Lazy`?

By default, singleton beans are generally created during startup.

Java

```java
@Service
public class PaymentService {
}
```

So:

```
Application startup
       ↓
PaymentService created
```

With:

Java

```java
@Lazy
@Service
public class PaymentService {
```

```
}
```

Spring can defer creation until the bean is actually needed.

```
Application startup
       ↓
PaymentService NOT created
       ↓
First request for PaymentService
       ↓
PaymentService created
```

This is important when discussing startup performance.

---

19. Important: Bean Creation Order

Don't think of startup as simply:

```
Create A
Create B
Create C
Create D
```

Spring primarily follows **dependency relationships**.

If:

```
A → B → C
```

then Spring must resolve the required dependencies appropriately.

For example:

‹/› Java

```java
@Service
class A {
    A(B b) {}
}
```

‹/› Java

```java
@Service
class B {
    B(C c) {}
}
```

```java
@Service
class C {
}
```

Conceptually:

```
C
↓
B
↓
A
```

The exact creation process can be more complex because Spring also handles:

- Singleton caching
- Circular dependencies
- Lazy beans
- Factory methods
- `FactoryBean`
- Proxies
- `@DependsOn`
- BeanPostProcessors

---

20. Startup vs Request-Time Bean Creation

For normal singleton beans:

```
Application startup
       ↓
Bean created
       ↓
Stored in singleton cache
       ↓
Application ready
```

Later:

```
HTTP Request
     ↓
Controller
     ↓
Service
     ↓
```

```
Existing singleton bean
```

Spring generally doesn't create the singleton from scratch for every request.

For prototype beans:

```
getBean()
   ↓
New instance
```

Each lookup can result in a new instance.

---

21. Complete Startup Picture

For interview purposes, remember this flow:

```
                SpringApplication.run()
                         |
                         ↓
                Create ApplicationContext
                         |
                         ↓
                Prepare Environment
                         |
                         ↓
              Process Configuration
                         |
                         ↓
                Component Scanning
                         |
                         ↓
                Create BeanDefinitions
                         |
                         ↓
               Register BeanDefinitions
                         |
                         ↓
            BeanFactoryPostProcessors
                         |
                         ↓
               Register BeanPostProcessors
                         |
                         ↓
           Create Non-Lazy Singletons
                         |
                         ↓
                Resolve Dependencies
                         |
```

```
                ↓
 Dependency Injection
                |
                ↓
Bean Initialization
                |
                ↓
   AOP / Proxies
                |
                ↓
Context Refresh Complete
                |
                ↓
Application Started
                |
                ↓
Application Ready
```

Key interview points

At your experience level, be clear about these distinctions:

- `SpringApplication.run()` bootstraps Spring Boot.
- `ApplicationContext` is the main IoC container.
- **Component scanning** discovers components.
- **BeanDefinition** is metadata; it is not the actual bean object.
- **BeanDefinition registration** tells the container what beans exist.
- **BeanFactoryPostProcessor** works with bean metadata before bean creation.
- **Dependency resolution + injection** happens during bean creation.
- **BeanPostProcessor** processes bean instances.
- **Initialization callbacks** prepare the bean for use.
- **Singleton beans** are generally created eagerly during startup unless lazy.
- `ApplicationReadyEvent` indicates that application startup is complete and the application is ready.